Project:
Geavanceerde Computerarchitectuur

Sien Peerlinck & Thibaut Dalez

May 15, 2024

1 Introduction

For the project of this lab we had to choose something that is interesting and where parallel processing is an added value. Siens thesis on lossless data compression on fpga for the VenSpec-H spectrometer came into consideration. By implementing the same algorithm on GPU instead of on FPGA and using the same data, a comparison on processing speed and power can be made.

In section 2 the context of the project and the data is given. Section 3 explains the FELICS algorithm and in section 4, this algorithm is converted into a functioning program. With section 5 we end with a conclusion.

2 Context

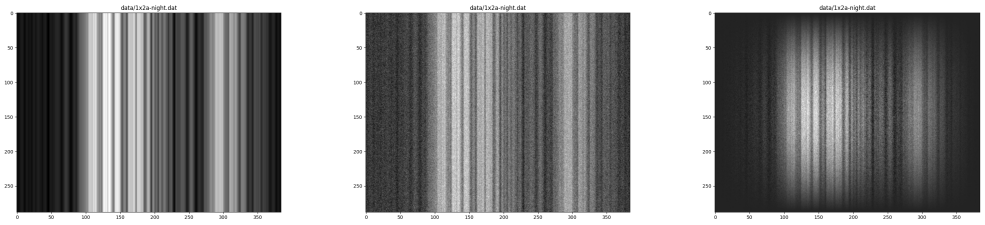
ESA's EnVision mission to Venus aims to investigate the planet's history and climate [1]. Despite its similarities to Earth in terms of proximity and size, Venus exhibits extreme conditions with surface temperatures averaging 464 °C and atmospheric pressure about 90 times higher than Earth's [2]. The planet's atmosphere, composed mostly of CO_2 with clouds of sulfuric acid, suggests a past less hostile than the present. Scientists seek to understand when and why Venus' path diverged from Earth's, particularly the role of geological and volcanic activity in triggering its extreme greenhouse effect.

Developed by the Royal Belgian Institute for Space Aeronomy (BIRA), VenSpec-H uses infrared spectrometry to detect volcanic activity traces in Venus' lower atmosphere and above its cloud deck. Spectrometry involves analyzing light absorption by molecules [3], with VenSpec-H focusing on wavelengths between 1 µm and 2.7 µm, divided into four spectral bands. The instrument, which includes a cooled optical bench, filter wheel, and detector, operates in varying temperature ranges to minimize noise from its own thermal emission.

Transmitting data from EnVision's instruments to Earth is challenging due to the relative positions of Earth, Venus, and the sun, which affect downlink capacity [1]. The greater the distance between Earth and Venus, the lower the downlink potential, with solar interference further reducing available bandwidth. To manage data transmission without quality loss, lossless data compression is employed.

A theoretical representation of the spectral data (figure 1a) shows each horizontal line as an absorption line, with pixel intensity indicating absorption levels. The frame resolution is 384x288 pixels, with varying noise levels depending on the spectral band and measurement conditions (day or night side of Venus). The effect of this noise is shown in figure 1b. Edge effects can also be observed at the borders of the image. For instance, the intensity in the corners of the frame will decrease compared to the center (vignetting). Manufacturing defects and cosmic radiation affecting the sensor system

can also cause bad pixels, which may show a zero value, the maximum value, or remain stuck at an intermediate value. The influence of vignetting and the dead pixels is shown in figure 1c.



(a) Simulated data. (b) after adding of noise. (c) after vignetting and dead pixels.

Figure 1: Simulation of the data (spectral band 1, nightside).

3 Solution

Fast and Efficient Lossless Image Compression (FELICS) is a technique proposed by Paul G. Howard and Jeffrey Scott Vitter as a simpler and faster method for lossless image compression [4]. In this algorithm, each pixel is assigned a code based on the ratio of its intensity relative to selected neighboring pixels. The neighbors of pixel P are chosen differently based on five different scenarios,

P_v	P_w			N2	N1	P_x	
N2	N1						
P_y							
					N1		
				N2	P_z		

Figure 2: Selection of the neighbours N1 and N2 in 5 scenerio's [5].

illustrated in figure 2. The first two pixels in the top row (P_v and P_w) have no neighbors and are not adjusted during encoding. The distribution of pixel intensities in an image is shown in figure 3, where L indicates the lower neighboring intensity value and H the higher one. There are three possible cases for pixel intensity:

(a) In-Range: If the intensity of pixel P lies within the range [L, H], one bit (0) is used to indicate this in-range position. Since the values of P are almost uniformly distributed within this range, P is assigned an adjusted binary code that is slightly shorter in the middle of this area [5]. The value coded is ε = P - L.

(b) Below-Range: If the intensity of pixel P is lower than L, one bit (1) indicates this out-of-range position, followed by one bit (0) to specify the below-range position. The probability of

out-of-range values decreases significantly the further they are from the range, allowing the use of exponential prefix codes such as Golomb or Rice code. Sub-exponential codes are also appropriate here. The value coded is ε = L - P.

(c) Above-Range: If the intensity of pixel P is higher than H, one bit (1) indicates this out-of-range position, followed by one bit (1) to specify the above-range position. Again, sub-exponential codes are used. The value coded is ε = P - H.

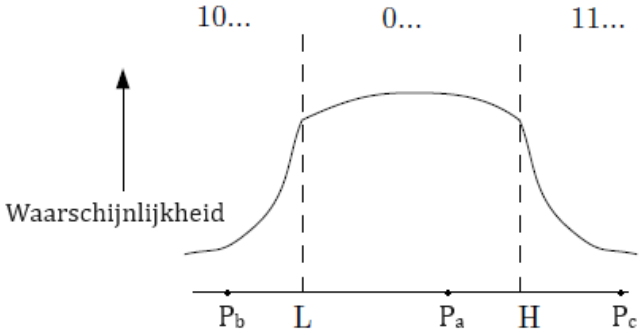


Figure 3: Intensity distribution of the pixel P.

3.1 Adjusted Binary Code

For the in-range codes, adjusted binary coding is used. This coding technique, also known as phased-in coding or truncated binary coding, is suitable for data where the individual symbols are equally likely or almost equally likely [6]. In this technique, some symbols are represented with one bit less than usual. The number of such symbols depends on the number of symbols that need to be represented; in the case of FELICS, this is the difference between the two neighbors (delta). Given the slightly higher probability of pixels whose value lies in the middle of the [L, H] interval, this implementation ensures that those pixels receive the shorter codes. Below is a detailed example for all possible symbols when delta=10.

1. H-L = delta = 10
2. with delta = 10, ε can take on 11 different values (0, 1, ...10) $\Rightarrow$ range = 11
3. To represent these 11 values in binary, you need 4 bits $\Rightarrow$ m = 4
4. This determines a *threshold*, which is the number of symbols that will be represented with a shorter codeword (m-1 bits) $\Rightarrow$ $threshold = 2^m - range = 2^4 - 11 = 5$
5. shift = $\frac{range - m}{2}$
6. Based on these parameters, the codewords can now be determined.
   (a) The first *shift* symbols are represented as ε + *threshold* with a 1 appended to avoid confusion[1]. These codewords are thus *m* bits long.
   (b) The last *shift* symbols are represented as delta - ϵ + threshold with a 0 appended. These codewords are also *m* bits long.
   (c) The remaining symbols (the middle *threshold*) are represented as ε - *shift*. These codewords thus count *m*-1 bits, which ensures that the average length of a codeword is shorter.

[1]The codewords for the first and last group of symbols are symmetrical. To distinguish them, a bit (0 or 1) is added

Table 1: Example of adjusted binary code for delta = 10

ε	calculation	result	codeword	number of bits used
0	ε + threshold	11	1011	4
1	ε + threshold	13	1101	4
2	ε + threshold	15	1111	4
3	ε - shift	0	000	3
4	ε - shift	1	001	3
5	ε - shift	2	010	3
6	ε - shift	3	011	3
7	ε - shift	4	100	3
8	delta - ε + threshold	14	1110	4
9	delta - ε + threshold	12	1100	4
10	delta - ε + threshold	10	1010	4

3.2 Sub-Exponential Code

For out-of-range symbols, the sub-exponential code is used, this technique is related to Rice codes [6]. This code uses a parameter k. For $\varepsilon < 2^{k+1}$, the length of the resulting codeword increases linearly with k, while for larger ϵ, the length increases logarithmically. To derive the codeword, two additional parameters, u and b, are first calculated:

$$b = \begin{cases} k, & \varepsilon < 2^k \\ \lfloor \log_2 \varepsilon \rfloor, & \varepsilon \geq 2^k \end{cases} \quad en \quad u = \begin{cases} 0, & \varepsilon < 2^k \\ b - k + 1, & \varepsilon \geq 2^k \end{cases}$$

The codeword then consists of a unary code of u and the b least significant bits of ε.

Some examples:

- $\varepsilon = 17455, k = 12 \Rightarrow b = 14\ en\ u = 3 \Rightarrow \underbrace{1110}_{\text{unary u}}\ \underbrace{00010000101111}_{\text{b lsb van } \varepsilon}$
- $\varepsilon = 10, k = 3 \Rightarrow b = 3\ en\ u = 0 \Rightarrow 0\ 010$

4 Code

In this section the code of the project will be explained. First, some changes to the algorithm are discussed. Then, different parts of the implementation will be analyzed in detail.

4.1 Changes to the algorithm

To prepare the algorithm for parallelism, it needed to slightly change. The dependency between horizontal lines of the image was removed so that all lines can now be processed together. To achieve this, the selection of neighbors is done differently: instead of always taking the pixel before and above as neighbors, now only the two preceding pixels are considered, as was previously the case in the top row (see P_x in figure 2). Therefore, the first two pixels of each row will not be compressed.

4.2 Felics Kernel

The Felics code functions as the Cuda kernel that is called when executing the Main code. First, it determines the difference between the first 2 pixels of a row in the input array (these are not compressed so they are passed immediately to the output_array). All the other pixels can be processed simultaneously because the algorithm uses the uncompressed preceding pixels to compress a pixel, this way a lot of parallelism is introduced.

4.3 Compress Function

To implement the other functions and to keep it clear we used __device__ functions.

```
__device__ int* compress(uint8_t N1, uint8_t N2, uint8_t P, int* res_str)
```

These are normal functions but with the added __device__ (or __host__) in front of them they are only usable by the GPU (or CPU). The compress code is the function that compresses the data (with the help of ABC and SEC). First it calculates the higher and lower bound and it uses these bounds to decide whether to use ABC or SEC. After this, a prefix is added (this prefix consists of '00', '10' or '11') that is used for decoding. Because of the '00' prefix we convert the pixel into binary (if we use an e.g. an integer, the 00 prefix would disappear and it wouldn't be possible to distinguish it from other codes, e.g. if the pixel would start with '11').

4.4 Adjusted Binary Code

In this device function the adjusted binary code is calculated for the case of an in-range pixel. First, as presented in section 3.1, the parameters range, upper_bound, threshold and shift need to be calculated. After that, the codeword is calculated and the result is returned.

4.5 Sub-exponential code

For the sub-exponential code, also a device function, the parameters b and u need to be calculated. For that, the parameter k is taken as a constant 12 for every image. This value has proven to be the most effective for the test data and by using a constant value, the need to calculate the most optimal one was removed, reducing the computational complexity of the design. After b and u are calculated according to section 3.2, an array of the size u+b+1 is created to store the intermediate results, as this will be the size of the final codeword. The first u elements of the array are set to 1, and the u'th element is 0. Then, the value that needed coding is converted into its binary form using the helper function toBinary() and the last b bits of this binary representation are added to the array. This array of ones and zeros is then converted to an integer and returned to the compress function.

5 Conclusion

In this project, we explored the implementation of the FELICS algorithm for lossless data compression on GPU, aiming to optimize the data processing speed and power efficiency for the VenSpec-H spectrometer, a key instrument in ESA's EnVision mission to Venus.

The FELICS algorithm proved to be highly adaptable for parallel processing on a GPU. By modifying the algorithm to eliminate dependencies between horizontal lines in the image, we achieved significant parallelism, allowing multiple pixels to be processed simultaneously. This modification not only facilitated the GPU implementation but also maintained the integrity and efficiency of the compression algorithm.

Sadly, our implementation of the algorithm on GPU never worked. This is likely due to an incorrect use of memory between the cpu and gpu that caused the array_output to only return zero values. We think that this is the cause because when we wrote larger amounts of data to the allocated memory the zero values appeared, this was not the case when small amounts of data (simple integers) were assigned to output_array.

A Program

The code can be found at https://github.com/SienPeerlinck/Geavanceerde-Computerarchitectuur/blob/main/project_Sien_Thibaut.cu

References

[1] "ESA Science & Technology - EnVision Assessment Study Report (Yellow Book)," Tech. Rep., Feb. 2021, p. 111. [Online]. Available: https://sci.esa.int/web/cosmic-vision/-/envision-assessment-study-report-yellow-book (visited on 12/17/2023).

[2] A. Observatory and Planetarium, *Venus: Earth's Sister Planet – Astronotes*, en-GB, Jan. 2012. [Online]. Available: https://armaghplanet.com/venus-earths-sister-planet.html (visited on 01/29/2024).

[3] *Infrared spectroscopy*, en, Page Version ID: 1199317713, Jan. 2024. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Infrared%5C_spectroscopy%5C&oldid=1199317713 (visited on 01/30/2024).

[4] P. Howard and J. Vitter, "Fast and efficient lossless image compression," in *[Proceedings] DCC '93: Data Compression Conference*, Mar. 1993, pp. 351–360. DOI: 10.1109/DCC.1993.253114. [Online]. Available: https://ieeexplore.ieee.org/document/253114 (visited on 11/01/2023).

[5] D. Salomon and G. Motta, "Image Compression," en, in *Handbook of Data Compression*, D. Salomon and G. Motta, Eds., London: Springer, 2010, pp. 443–730, ISBN: 978-1-84882-903-9. DOI: 10.1007/978-1-84882-903-9_7. [Online]. Available: https://doi.org/10.1007/978-1-84882-903-9%5C_7 (visited on 11/01/2023).

[6] D. Salomon and G. Motta, "Advanced VL Codes," en, in *Handbook of Data Compression*, D. Salomon and G. Motta, Eds., London: Springer, 2010, pp. 95–176, ISBN: 978-1-84882-903-9. DOI: 10.1007/978-1-84882-903-9_3. [Online]. Available: https://doi.org/10.1007/978-1-84882-903-9%5C_3 (visited on 05/04/2024).